

ExoHabitAI Project Documentation

Frontend

The frontend of the ExoHabitAI system provides a user-friendly interface that allows users to input planetary and stellar parameters and view habitability predictions.

Technologies Used

- HTML5 – Page structure
- CSS3 – Styling and layout
- Bootstrap – Responsive UI design
- JavaScript – Client-side logic and API communication

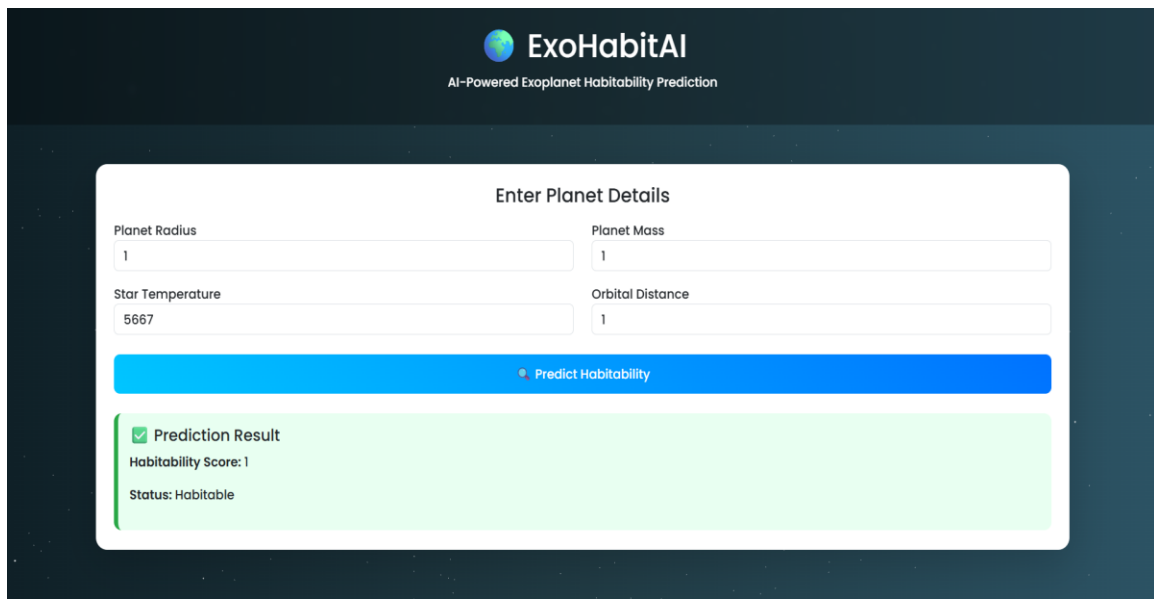
Frontend Features

- Input form for planet radius, mass, star temperature, and orbital distance
- Client-side validation for numeric values
- Fetch API used to send data to backend
- Dynamic display of habitability score and status
- Professional UI with gradient background and card layout

Frontend Workflow

User enters parameters → Frontend validates inputs → Data sent to backend → Backend returns prediction → Result displayed on UI

Output



The screenshot displays the ExoHabitAI web application interface. At the top, the logo "ExoHabitAI" is shown with the tagline "AI-Powered Exoplanet Habitability Prediction". Below this is a form titled "Enter Planet Details". The form contains four input fields: "Planet Radius" (value: 1), "Planet Mass" (value: 1), "Star Temperature" (value: 5667), and "Orbital Distance" (value: 1). A blue button labeled "Predict Habitability" is positioned below the input fields. Below the button, a green box displays the "Prediction Result" with a checkmark icon, showing a "Habitability Score: 1" and a "Status: Habitable".

Backend

The backend is built using Flask and integrates a trained machine learning model to predict the habitability of exoplanets based on input parameters.

Technologies Used

- Python
- Flask framework
- Scikit-learn (Machine Learning)
- Joblib/Pickle for model loading
- NumPy/Pandas for data handling

Backend Components

- Flask server to handle requests
- Model loading during server startup
- /predict API endpoint for receiving input
- Data processing and feature formatting
- Model prediction and probability scoring
- JSON response returned to frontend

Backend Workflow

Frontend sends JSON request → Flask receives request → Data processed → ML model predicts → JSON response returned

System Integration Flow

User → Frontend Form → Fetch API → Flask Backend → Machine Learning Model → Prediction Output → UI Display